

1. Investigación técnica (Defensa TFG): modelos adicionales, hiperparámetros y validación

1.1. 1) Alcance

Este documento complementa al informe de arquitectura y **no** desarrolla de nuevo:

- esquema canónico (ya documentado)
- algoritmo QRDQN en detalle (ya documentado)

Aquí se resumen: otros modelos/algoritmos, parámetros y sistema de evaluación.

1.2. 2) Modelos y algoritmos presentes además del núcleo QRDQN

1.2.1. 2.1 Baseline clásico: Random Forest

Archivo: `src/baseline_random_forest.py`

- Modelo: `RandomForestClassifier`
- Hiperparámetros por defecto:
 - `n_estimators=200`
 - `max_depth=None`
 - `n_jobs=-1`
 - `random_state=42`
- Se puede ejecutar sobre CICIDS2017 o NSL-KDD (histórico)
- Evalúa con `confusion_matrix` y `classification_report`

Uso en defensa: baseline no-RL para comparar la estrategia RL contra un método supervisado clásico.

1.2.2. 2.2 DQN como fallback operativo

Archivo: `scripts/predict_real_traffic_v2.py` (`load_model`).

- Intenta cargar con `sb3_contrib.QRDQN`
- Si ocurre una excepción durante esa carga, hace fallback a `stable_baselines3.DQN`

Esto da robustez operativa de inferencia cuando falla la carga de QRDQN; la ausencia de `sb3_contrib` es un caso posible, pero no el único.

1.2.3. 2.3 Búsqueda de hiperparámetros con Optuna

Archivo: `src/tune_hparams.py`

Espacio de búsqueda:

- `learning_rate`: `1e-5 ... 1e-3` (log)
- `batch_size`: `{256, 512, 1024, 2048}`
- `gradient_steps`: `{10, 50, 100}`
- `net_arch`: `256_128, 512_256, 256_256`
- `gamma`: `0.95 ... 0.999`
- `train_freq`: `{50, 100, 200}`

Objetivo optimizado: F1 de la clase ataque (`pos_label=1`).

1.3. 3) Arquitecturas de red usadas en el proyecto

Aunque la investigación específica de QRDQN está en otro documento, en el código aparecen arquitecturas MLP concretas:

- `net_arch=[512, 256]`
 - entrenamiento principal (`src/train_rl_defender.py`)
 - validación Check C y leave-one-CSV-out
- `net_arch=[256, 128]`
 - validación Check B (labels barajadas)

Interpretación para defensa:

- la arquitectura más grande (512,256) se reserva para entrenamiento principal y validación exigente
- la más pequeña (256,128) acelera el test anti-leakage

1.4. 4) Hiperparámetros clave de entrenamiento y validación

1.4.1. 4.1 Entrenamiento principal (`src/train_rl_defender.py`)

Comunes:

- `learning_rate=1e-4`
- `gamma=0.99`
- `tau=1.0`
- `buffer_size=min(200_000, max(total_timesteps, 10_000))`

Dependientes del preset:

- **fast**
 - `batch_size=512`
 - `gradient_steps=10`
 - `train_freq=50`
 - `target_update_interval=1_000`
 - `total_timesteps` default: 25_000
- **full**
 - `batch_size=2048`
 - `gradient_steps=20`
 - `train_freq=100`
 - `target_update_interval=10_000`
 - `total_timesteps` default: 100_000

Split de datos:

- `split_mode=random` o `split_mode=day`
- presets de filas (`fast` vs `full`) vía `load_cicids2017_split`

1.4.2. 4.2 Leave-one-exact-CSV-out (`src/validate_leave_one_csv_out.py`)

- `timesteps` por fold: default 30_000
- `predict_batch_size` default 8192
- entrenamiento por fold con `net_arch=[512,256]`, `batch_size=512`, `gradient_steps=20`, `train_freq=100`
- agrega métricas por fold y globales

1.5. 5) Métricas y validación experimental

1.5.1. 5.1 Check A (evaluación directa)

Archivo: `src/validate_checks.py`, función `check_a_direct_eval`

- Predicción directa `model.predict(X_test[i])`
- Reporta: accuracy, precision/recall/F1 para ataque y benigno, matriz de confusión y TP/FP/FN/TN

Objetivo: validar rendimiento sin depender de señales auxiliares del entorno.

1.5.2. 5.2 Check B (anti-leakage con etiquetas barajadas)

Archivo: `src/validate_checks.py`, función `check_b_shuffled_labels`

- baraja `y_train`
- reentrena brevemente
- compara accuracy vs baseline de clase mayoritaria
- regla: `leakage_detected = shuffled_acc > baseline_acc + 0.05`

Objetivo: comprobar que el rendimiento alto desaparece al romper la relación real X-y.

1.5.3. 5.3 Check C (split por día/CSV)

Archivo: `src/validate_checks.py`, función `check_c_csv_split`

- entrena en un conjunto de días/CSV
- evalúa en días/CSV diferentes
- reporta mismas métricas de clasificación + conteos

Objetivo: medir generalización fuera del split aleatorio.

1.5.4. 5.4 Leave-one-exact-CSV-out

Archivo: `src/validate_leave_one_csv_out.py`

- cada fold deja un CSV oficial completo para test
- agrega: medias/desviaciones/min/max por métrica, matriz global agregada, métricas globales (accuracy, balanced accuracy, F1, FPR/FNR, reward)

Es la validación más estricta a nivel de separación por archivo real.

1.6. 6) Parámetros de recompensa (impacto en comportamiento)

Valor por defecto común en entrenamiento/validaciones:

- `tp=1.5`
- `fp=-1.5`
- `fn=-5.0`
- `omission=0.0`

Lectura para defensa:

- se penaliza más el falso negativo que el falso positivo
- por diseño, el agente está sesgado a evitar dejar pasar ataques

1.7. 7) Qué puede preguntarte el tribunal y cómo enlazarlo

1. “¿Cómo verificas que no hay leakage?” — Check B (labels barajadas) + política explícita de drop de columnas de identificación.
2. “¿Cómo pruebas generalización realista?” — Check C y leave-one-exact-CSV-out.
3. “¿Qué alternativa no-RL tienes?” — baseline Random Forest.
4. “¿Cómo soportas inferencia robusta en datos reales?” — pipeline v2 con clipping + diagnósticos de drift + artefactos reproducibles.